説明資料：Q-Learning による ROS 2 タートルの自律目標到達

> 提出前に、角括弧内を実測した内容に置き換える。未実施の数値は記載しない。

1. 課題の概要と目的

ROS 2 のロボットシミュレータ turtlesim 内で、タートルが指定した目標座標へ自律移動するタスクを実装した。目的は、ロボットが得る位置・姿勢の観測から、AIが行動を選択し、報酬を通じて目的達成行動を獲得する流れを小さな構成で確認することである。

2. システム構成

/turtle1/pose からタートルの位置と姿勢を取得する。観測を「目標までの距離」と「目標方向との角度差」に離散化し、Q表の状態として利用する。Q-learning は 前進、左旋回、右旋回 から epsilon-greedy 法で行動を選び、/turtle1/cmd_vel に速度指令を送る。

報酬は、前ステップより目標へ近づけば正、離れれば負とした。目標到達には大きな正の報酬、壁際への接近と時間経過には負の報酬を与えた。この設計により、到達しやすさと安全な移動を同時に学習対象にした。

3. AIの利用内容

本課題では、Q-learning を方策として用いた。各状態と行動の価値を Q 表に保存し、学習時には探索を含む行動選択、評価時には最大 Q 値の行動選択を行う。目標方向へ向ける処理や軌道を固定的に記述せず、行動選択は学習済み Q 値に基づく。

4. 実装上の工夫

- 連続値の位置・姿勢を、距離6区分と方位誤差8区分へ離散化し、小さなQ表で学習できるようにした。
- 1ステップごとの距離変化を報酬にし、到達だけでなく途中の改善も学習できるようにした。
- 壁際に近づく行動にペナルティを与え、シミュレータの境界へ張り付く失敗を減らす設計とした。
- Q表をJSONで保存し、学習後に評価モードで同じ方策を再現できるようにした。

5. 実験条件と結果

RUNBOOK.md の generate_report.py が出力する REPORT_MEASUREMENTS.md から、下表の実測項目を転記する。実行していない項目は空欄のままにする。

項目	内容
ROS 2 ディストリビューション	[例：Humble]
目標座標	[実測・設定値]
学習エピソード数	[実行値]
乱数シード	[実行値]
評価試行数	[実行値]
到達数 / 成功率	[実測値]
評価時の所要時間	[実測値]

実行記録: 学習 [runs/<train-run-id>/]、評価 [runs/<eval-run-id>/]。評価の各試行結果は evaluation_trials.jsonl を付録または本文の要約として残す。

評価はRUNBOOKの評価コマンドで10試行を連続実行する。各試行の前に開始位置へ戻し、同じQ表・目標座標・seedを使う。各試行の到達/timeoutは evaluation_trials.jsonl に記録する。

6. 考察と今後の改善

離散化が粗すぎる場合は、目標近傍で旋回を繰り返す可能性がある。距離・角度の区分、報酬、探索率を調整して改善できる。次の段階では、壁との距離を観測に加えること、複数の開始位置・目標位置で訓練すること、深層強化学習へ置き換えることを検討する。

7. 実行状況

ROS 2パッケージのコードと手順を作成した段階であり、この草稿作成環境ではROS 2 build、turtlesim実行、学習収束、評価成功率を確認していない。実行後にROS 2版、学習ログ、評価試行数、到達数、実行動画を記入する。未実行の数値を実績として記載しない。

8. 提出物

- GitHubリポジトリ：［公開URL］
- YouTube動画：［限定公開または公開URL］
- 動画再生速度：［等速／2倍速。2倍速の場合は明記］

9. 参考資料

- ROS 2 turtlesim package documentation: docs.ros.org